Advanced Encryption Standard (AES): From Abstract Algebra to Complete Encryption Process

Understanding AES through Mathematics and Complete Encryption Process

B.TECH CS / IT | CRYPTOGRAPHY AND NETWORK SECURITY

Mathematical Foundations

Group → Ring → Field → GF(2^8) — the algebraic bedrock of every AES operation

Complete AES Architecture

10 rounds of SubBytes, ShiftRows, MixColumns, AddRoundKey — traced byte by byte

NIST FIPS 197 Test Vectors

Official plaintext, key, and ciphertext used throughout — every calculation verified

Learning Objectives

Foundations

- Understand the **need for AES** and its advantages over legacy ciphers like DES
- Master **Mathematical Foundations**: Group → Ring → Field → Finite Field hierarchy
- Understand **Galois Field GF(2^8)** and polynomial arithmetic over binary fields
- Analyze **AES Architecture**: 128-bit block, key sizes, round structure
- Perform **Key Expansion**: RotWord, SubWord, Rcon, round key generation

Operations & Applications

- Execute all four round operations: **SubBytes, ShiftRows, MixColumns, AddRoundKey**
- Trace a **Complete Encryption** (Round 0 → Round 10) with full state matrices
- Understand **Complete Decryption** using inverse operations in reverse order
- Compare **DES vs AES** on security, performance, and key length
- Identify **Real-World Applications**: SSL/TLS, VPN, Banking, Wi-Fi, Cloud

History of AES: From DES Failure to Global Standard

1

1977 — DES Adopted

56-bit key, 64-bit block; became the first widely-used symmetric cipher

2

1997 — DES Broken

RSA DES Challenge cracked in 96 days; 56-bit key space exhausted by brute force

3

1997 — NIST Call

Open competition for a new AES; 15 candidates submitted globally

4

1999 — 5 Finalists

MARS, RC6, Rijndael, Serpent, Twofish evaluated for security, efficiency, flexibility

5

2000 — Rijndael Selected

Designed by Joan Daemen & Vincent Rijmen (Belgium); best balance of security + speed

6

2001 — FIPS 197

AES officially standardized; supports 128/192/256-bit keys

7

2003 — NSA Approval

Cleared for TOP SECRET classification; first public cipher for classified data

8

Today

Global standard — TLS 1.3, WPA3, AES-NI hardware acceleration in every modern CPU

CHAPTER 2

Mathematical Foundations

Group · Ring · Field · Finite Field — the algebraic hierarchy powering every AES operation

Group One operation, four axioms	**Ring** Two operations, no inv. req.
Field Ring + multiplicative inverse	**GF(2^8)** 256-element finite field for AES bytes

Why Mathematics Powers AES

AES is Not Heuristic — It Is Algebraic

Every operation in AES is mathematically defined and provably secure. The entire cipher is built on a strict hierarchy of algebraic structures:

Numbers → Groups → Rings → Fields → Finite Fields → GF(2^8) → AES

Security is not empirical guesswork — it is a consequence of the algebraic properties of GF(2^8).

How Each Operation Maps to Mathematics

SubBytes

Multiplicative inverse in GF(2^8) — requires a field

MixColumns

Polynomial multiplication mod the AES irreducible polynomial in GF(2^8)

AddRoundKey

XOR = addition in GF(2), the simplest finite field

Why Finite Fields?

Operations stay within 0–255; exact, reversible, no overflow

Security Guarantee

Algebraic structure ensures diffusion and confusion — Shannon's two principles

Group: Definition and Properties

Formal Definition

A **Group (G, ★)** is a set G with a binary operation ★ satisfying exactly four axioms:

1

Closure

$\forall\ a, b \in G \rightarrow a \bigstar b \in G$
(result stays inside the set)

2

Associativity

$\forall\ a,b,c \in G \rightarrow (a \bigstar b) \bigstar c = a \bigstar (b \bigstar c)$

3

Identity

$\exists\ e \in G : a \bigstar e = e \bigstar a = a$ for all a

4

Inverse

$\forall\ a \in G, \exists\ a^{-1} \in G : a \bigstar a^{-1} = e$

Concrete Examples

$(\mathbb{Z}, +)$

Integers under addition; identity = 0; inverse of 5 = −5; infinitely many elements

$(\mathbb{Z}_5, +)$

{0,1,2,3,4} under addition mod 5; closed, associative, identity=0, every element has inverse

Every finite field GF(2^8) forms a group under both addition and multiplication — the group axioms are the prerequisite for field structure.

Group Example: $(\mathbb{Z}_5, +)$ Step-by-Step

Worked Calculations

Set: $\mathbb{Z}_5 = \{0, 1, 2, 3, 4\}$; Operation: Addition modulo 5

Example — 2 + 4 mod 5:

- Step 1: 2 + 4 = 6
- Step 2: 6 mod 5 = **1** $\in \mathbb{Z}_5$ ✔️ (Closure satisfied)

Identity check: 3 + 0 = 3 mod 5 = 3 ✔️

Inverse of 3: 3 + 2 = 5 mod 5 = 0 ✔️ → inverse of 3 is **2**

Associativity: (1+2)+4 = 7 mod 5 = 2; 1+(2+4) = 1+1 = 2 ✔️

$\mathbb{Z}_5$ Addition Table

+	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

Every row and column contains each element exactly once — confirming closure and the existence of inverses.

Abelian Group: Commutativity and Cayley Tables

Definition and Key Properties

A group $(G, \bigstar)$ is **Abelian (commutative)** if $\forall\ a, b \in G$: $\mathbf{a \bigstar b = b \bigstar a}$

Abelian Example

$(\mathbb{Z}_5, +)$: 2+3 = 5 mod 5 = 0; 3+2 = 5 mod 5 = 0 ✔️ → Abelian

Non-Abelian Example

Matrix multiplication: AB ≠ BA in general → not commutative

Cayley Table: A complete multiplication table showing all pairwise results. Symmetric about the diagonal for Abelian groups — a quick visual test for commutativity.

Relevance to AES: $GF(2^8)$ addition (XOR) is Abelian — commutative and associative, forming the additive group of the field.

Cayley Table for $(\mathbb{Z}_4, +)$

★	0	1	2	3
0	0	1	2	3
1	1	2	3	0
2	2	3	0	1
3	3	0	1	2

The table is symmetric about the main diagonal — confirming commutativity of $(\mathbb{Z}_4, +)$.

Ring: Two Operations, Extended Structure

Formal Definition

A **Ring (R, +, ×)** is a set with two binary operations satisfying:

- **(R, +) is an Abelian Group:** closure, associativity, identity (0), inverse, commutativity
- **Multiplication is Associative:** (a×b)×c = a×(b×c)
- **Distributive Laws:** a×(b+c) = a×b + a×c and (a+b)×c = a×c + b×c

Key distinction from Field: A ring does NOT require multiplicative inverses for all non-zero elements.

Ring Examples and the Critical Boundary

$(\mathbb{Z}, +, \times)$

Integers — addition and multiplication both defined; no multiplicative inverse for most elements (e.g., $5^{-1} \notin \mathbb{Z}$)

Polynomial Ring

All polynomials with integer coefficients; closed under + and ×

$\mathbb{Z}_6$ — NOT a Field

$2 \times 3 = 6 \bmod 6 = 0 \rightarrow$ zero divisors exist; 2 and 3 have no multiplicative inverse

Ring: Addition and Multiplication Tables ($\mathbb{Z}_4$)

Set: $\mathbb{Z}_4 = \{0, 1, 2, 3\}$ under addition and multiplication mod 4. **Key observation:** $2 \times 2 = 4 \bmod 4 = 0 \rightarrow 2$ is a zero divisor $\rightarrow \mathbb{Z}_4$ is a Ring, **NOT a Field**.

$\mathbb{Z}_4$ Addition Table

+	0	1	2	3
0	0	1	2	3
1	1	2	3	0
2	2	3	0	1
3	3	0	1	2

$\mathbb{Z}_4$ Multiplication Table

×	0	1	2	3
0	0	0	0	0
1	0	1	2	3
2	0	2	0	2
3	0	3	2	1

Highlighted cell: $2 \times 2 = 0$ — zero divisor; element 2 has no multiplicative inverse $\rightarrow \mathbb{Z}_4$ cannot be a field.

Field: Every Non-Zero Element Has a Multiplicative Inverse

Formal Definition

A **Field (F, +, ×)** is a Ring where every non-zero element has a multiplicative inverse, and (F \ {0}, ×) is also an Abelian group.

Fields in Mathematics

- $\mathbb{R}$ (Real Numbers)
- $\mathbb{Q}$ (Rational Numbers)
- $\mathbb{C}$ (Complex Numbers)
- GF(p) for any prime p

Why Fields matter for AES: SubBytes requires computing multiplicative inverse in $GF(2^8)$ — only possible because $GF(2^8)$ is a field, not merely a ring.

Finding Inverses in GF(5): Step-by-Step

Example — Find 3^{-1} mod 5:

01

Try 3 × 1 = 3 mod 5 = 3 ⊡

Not the identity element

02

Try 3 × 2 = 6 mod 5 = 1 ✔️

$3^{-1} = 2$ in GF(5)

03

Verification

$3 \times 2 = 6 = 1 \bmod 5$ ✔️ — identity confirmed

All inverses in GF(5): $1^{-1}=1$, $2^{-1}=3$, $3^{-1}=2$, $4^{-1}=4$

Finite Field: Fixed Number of Elements

Existence Theorem

A finite field **GF(q)** exists if and only if $\mathbf{q = p^n}$ where p is prime and $n \geq 1$. The field has exactly q elements and all arithmetic stays within that set.

GF(2)

{0,1} — binary field; addition=XOR, multiplication=AND

GF(5)

{0,1,2,3,4} — prime field; arithmetic mod 5

GF(7)

{0,1,2,3,4,5,6} — prime field; arithmetic mod 7

GF(2^8)

256 elements — extension field used in AES for byte-level operations

GF(2) — The Simplest Finite Field

GF(2) +	0	1	GF(2) ×	0	1
0	0	1	**0**	0	0
1	1	0	**1**	0	1

GF(2) addition is XOR. GF(2) multiplication is AND. Every element is its own additive inverse: $1 \oplus 1 = 0$.

GF(2^8) is an **extension** of GF(2) — its 256 elements are polynomials with coefficients drawn from GF(2).

GF(5) Complete Tables

GF(5) = {0, 1, 2, 3, 4} — all arithmetic performed mod 5. Every non-zero element has a multiplicative inverse, confirming GF(5) is a field.

Multiplicative Inverses in GF(5): $1^{-1}=1$, $2^{-1}=3$, $3^{-1}=2$, $4^{-1}=4$ | **Verification:** $2\times3=6 \bmod 5=1$ ✔️; $4\times4=16 \bmod 5=1$ ✔️

GF(5) Addition Table

+	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

GF(5) Multiplication Table

×	0	1	2	3	4
0	0	0	0	0	0
1	0	1	2	3	4
2	0	2	4	1	3
3	0	3	1	4	2
4	0	4	3	2	1

CHAPTER 3

Galois Field GF(2^8)

The Heart of AES — 256-element field where every byte lives and every operation executes

256 Elements

$2^8 = 256$; one element per possible byte value 0x00–0xFF

Polynomial Representation

Each byte is a degree-7 polynomial with binary coefficients

Irreducible Modulus

$x^8+x^4+x^3+x+1$ = 0x11B — defines all field arithmetic

xtime Operation

Multiply by x: left-shift + conditional XOR 0x1B — basis of all GF multiplication

GF(2^8): Why AES Needs an Extension Field

The Core Requirement

AES operates on **bytes**: each byte = 8 bits = values 0–255. We need a field with exactly **256 elements** — that is GF(2^8).

GF(2^8) is an **extension field of GF(2)**. Elements are polynomials of degree ≤ 7 with coefficients in GF(2) = {0, 1}.

General element:
$b_7x^7 + b_6x^6 + b_5x^5 + b_4x^4 + b_3x^3 + b_2x^2 + b_1x + b_0$
where $b_i \in \{0,1\}$

Byte-to-Polynomial Conversion Examples

0x57 = 01010111_2

$x^6 + x^4 + x^2 + x + 1$

0x83 = 10000011_2

$x^7 + x + 1$

Addition in GF(2^8): XOR of the two bytes (coefficient addition mod 2 — no carry)

Multiplication in GF(2^8): Polynomial multiplication followed by reduction modulo the AES irreducible polynomial $m(x) = x^8+x^4+x^3+x+1$

Binary to Polynomial Representation

Conversion Rule: Each bit position i corresponds to the coefficient of x^i. Bit 7 is the most significant; bit 0 is the constant term.

0x57 → $x^6+x^4+x^2+x+1$

Binary: 0 1 0 1 0 1 1 1
Bits: 7 6 5 4 3 2 1 0
Active bits: 6,4,2,1,0 → **$x^6+x^4+x^2+x+1$**

0x83 → x^7+x+1

Binary: 1 0 0 0 0 0 1 1
Bits: 7 6 5 4 3 2 1 0
Active bits: 7,1,0 → **x^7+x+1**

0x01 → 1

Binary: 0 0 0 0 0 0 0 1
Only bit 0 active → **1** (multiplicative identity of GF(2^8))

0x02 → x

Binary: 0 0 0 0 0 0 1 0
Only bit 1 active → **x** (the generator element)

AES Irreducible Polynomial: $m(x) = x^8 + x^4 + x^3 + x + 1$

Why an Irreducible Polynomial?

To define $GF(2^8)$ as an extension field, we need a polynomial that **cannot be factored** over GF(2). This plays the same role as a prime modulus in $\mathbb{Z}_p$.

AES uses: $m(x) = x^8 + x^4 + x^3 + x + 1$

Hex: 0x11B | **Binary:** 100011011_2

When polynomial multiplication produces degree ≥ 8, reduce modulo m(x) to bring result back into $GF(2^8)$.

Key identity from reduction: $x^8 \equiv x^4 + x^3 + x + 1 \pmod{m(x)}$

Properties and Analogy

Irreducibility Verified

$m(0) = 1 \neq 0$; $m(1) = 5 \bmod 2 = 1 \neq 0$ → no roots in GF(2); NIST confirmed no quadratic/cubic factors

Why This Specific Polynomial?

NIST chose 0x11B for efficient hardware implementation and optimal diffusion properties in MixColumns

Analogy

Like working with complex numbers where $i^2 = -1$; here $x^8 = x^4+x^3+x+1 \pmod{m(x)}$

Addition in GF(2^8): XOR Operation

The Rule: Add Coefficients Mod 2

In GF(2^8), addition is simply **XOR** of the two bytes. Coefficients at each bit position add mod 2 — no carry is ever generated.

Example: 0x57 + 0x83

$0x57 = 01010111_2$
$0x83 = 10000011_2$
XOR: $11010100_2 = 0xD4$

Polynomial form:
$(x^6+x^4+x^2+x+1) + (x^7+x+1)$
$= x^7+x^6+x^4+x^2$ (x^1 and x^0 cancel mod 2)

Key Properties of GF(2^8) Addition

Addition = Subtraction

In GF(2^8), $-1 = 1 \bmod 2$, so $a + b = a - b$. Subtraction is the same operation.

No Carry — Stays in 8 Bits

Unlike integer addition, XOR never produces a carry. The result always fits in exactly 8 bits.

Self-Inverse Elements

Every element is its own additive inverse: $a \oplus a = 0$ for all a. This is why AddRoundKey is self-reversible.

Multiplication in GF(2^8): The xtime Operation

xtime(a): Multiplies a by x (i.e., by 0x02) — the fundamental building block of all GF(2^8) multiplication. All other multiplications are composed from repeated xtime calls.

Algorithm

1. Left-shift by 1 bit
2. If bit 7 of original *was* set → XOR result with `0x1B`
3. Mask to 8 bits

`0x1B` = 00011011_2 = x^4+x^3+x+1 (low 8 bits of m(x))

Example: xtime(0x57)

`0x57` = 01010111_2; bit7 = 0
Left shift → 10101110_2 = 0xAE
bit7 was 0 → no XOR → **result = 0xAE**

Example: xtime(0x83) — with reduction

01

0x83 = 10000011_2; bit7 = 1

Note: bit 7 is SET — reduction will be needed

02

Left shift → 00000110_2 = 0x06

The leading bit is lost (only 8 bits kept)

03

bit7 was 1 → XOR with 0x1B

`0x06 ⊕ 0x1B = 0x1D`

04

Result = 0x1D ✔️

Reduction complete — stays within GF(2^8)

GF(2^8) Multiplication: 0x57 × 0x83 Complete Example

Method: Expand 0x83 = x^7+x+1 and multiply 0x57 by each power using repeated xtime. Then XOR selected partial products.

xtime Chain for 0x57

Power	Hex	Binary
×0x01	0x57	01010111
×0x02	0xAE	10101110
×0x04	0x47	01000111
×0x08	0x8E	10001110
×0x10	0x07	00000111
×0x20	0x0E	00001110
×0x40	0x1C	00011100
×0x80	0x38	00111000

Final Assembly: 0x57 × 0x83

0x83 = 10000011_2 → active bits are at positions 7, 1, 0

Select partial products: ×0x80, ×0x02, ×0x01

0x57 × 0x80 = 0x38
0x57 × 0x02 = 0xAE
0x57 × 0x01 = 0x57

XOR all selected:
0x38 ⊕ 0xAE = 0x96
0x96 ⊕ 0x57 = 0xC1

0x57 × 0x83 = 0xC1 ✔️ — NIST FIPS 197 confirmed result

CHAPTER 3

Polynomial Arithmetic in GF(2^8)

Addition, multiplication, and modular reduction — all the algebra needed to understand every AES operation

Poly Addition

XOR coefficients — identical to byte XOR

Poly Multiplication

Distribute terms; accumulate with mod 2 coefficients

Modulo Reduction

Use $x^8 \equiv x^4+x^3+x+1$ to reduce degree ≥ 8 terms

Irreducibility

Guarantees every non-zero element has a unique inverse

Polynomial Addition and Subtraction in GF(2)

The Rule: Coefficient-wise XOR

Add/subtract coefficients mod 2. In GF(2), subtraction is **identical** to addition since $-1 \equiv 1 \pmod 2$.

Example: $(x^6+x^4+x^2+x+1) + (x^7+x+1)$

Power	Poly 1	Poly 2	Sum (mod 2)
x^7	0	1	1
x^6	1	0	1
x^4	1	0	1
x^2	1	0	1
x^1	1	1	**0** (cancel!)
x^0	1	1	**0** (cancel!)

Result and Verification

Polynomial result: $x^7 + x^6 + x^4 + x^2$

Binary: 11010100_2 = 0xD4

XOR verification:
0x57 = 01010111
0x83 = 10000011
XOR = 11010100 = 0xD4 ✔️

> The polynomial and XOR methods always agree — they are the **same operation** viewed differently. In implementation, always use XOR; in proofs, use polynomial notation.

Polynomial Multiplication: Step-by-Step

Multiply: $(x^6+x^4+x^2+x+1) \times (x^7+x+1)$ — distributing each term of the second polynomial across the first, then collecting coefficients mod 2.

Distribution Step

Distribute x^7: $x^{13} + x^{11} + x^9 + x^8 + x^7$

Distribute x: $x^7 + x^5 + x^3 + x^2 + x$

Distribute 1: $x^6 + x^4 + x^2 + x + 1$

Combining Terms (mod 2 coefficients)

Collect all terms and apply mod 2 to each coefficient:

x^7 appears twice: 1+1 = 0 (cancels)
x^2 appears twice: 1+1 = 0 (cancels)
x appears twice: 1+1 = 0 (cancels)

Unreduced result:
$x^{13} + x^{11} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + 1$

⚠ Degree is 13 — exceeds 7! Must reduce modulo $m(x) = x^8+x^4+x^3+x+1$ to bring back into $GF(2^8)$.

Modulo Reduction: Bringing Result Back to GF(2^8)

Key identity: $x^8 \equiv x^4+x^3+x+1 \pmod{m(x)}$. Apply this iteratively to reduce every term of degree ≥ 8.

01

Reduce x^{13}

$x^{13} = x^5 \cdot x^8 \equiv x^5(x^4+x^3+x+1) = x^9+x^8+x^6+x^5$

02

Reduce x^{11}

$x^{11} = x^3 \cdot x^8 \equiv x^3(x^4+x^3+x+1) = x^7+x^6+x^4+x^3$

03

Reduce x^9

$x^9 = x \cdot x^8 \equiv x(x^4+x^3+x+1) = x^5+x^4+x^2+x$

04

Reduce x^8

$x^8 \equiv x^4+x^3+x+1$ (direct substitution)

05

Collect All Terms mod 2

XOR all partial results → **Final result: 0xC1** ✔️ — matches xtime-based calculation

Irreducible Polynomial: Why It Cannot Be Factored

Definition and Test

A polynomial p(x) over GF(2) is **irreducible** if it has no roots in GF(2) and no non-trivial polynomial factors.

Testing $m(x) = x^8+x^4+x^3+x+1$

Test x=0: $m(0) = 0+0+0+0+1 = 1 \neq 0 \rightarrow 0$ is not a root ✔️

Test x=1: $m(1) = 1+1+1+1+1 = 5 \bmod 2 = 1 \neq 0 \rightarrow 1$ is not a root ✔️

No linear factors → must check for quadratic/cubic factors (all verified by NIST)

Why Irreducibility Matters

Field Construction

Ensures every non-zero element has a unique multiplicative inverse → $GF(2^8)$ is indeed a field

Other Options

Other irreducible polynomials of degree 8 over GF(2) exist — AES chose 0x11B for hardware efficiency

Prime Modulus Analogy

Just as choosing a prime modulus p makes $\mathbb{Z}_p$ a field, choosing an irreducible polynomial makes $GF(2^8)$ a field

CHAPTER 4

AES Architecture

128-bit blocks · Substitution-Permutation Network · 10, 12, or 14 rounds depending on key size

1

Plaintext

128-bit input block arranged as 4×4 state matrix

2

Key Schedule

Expand key into N+1 round keys

3

N Rounds

SubBytes → ShiftRows → MixColumns → AddRoundKey

4

Ciphertext

128-bit encrypted output block

AES High-Level Architecture

Core Properties

Block Cipher

Fixed 128-bit (16-byte) blocks of plaintext; block size never varies across AES-128/192/256

Symmetric Key

Same key used for both encryption and decryption; key size can be 128, 192, or 256 bits

SPN Structure

Substitution-Permutation Network alternating confusion (SubBytes) and diffusion (ShiftRows, MixColumns)

Round Structure Detail

Initial Round (Round 0)

AddRoundKey only — XOR plaintext state with the original key before any round operations

Rounds 1 to N−1

SubBytes → ShiftRows → MixColumns → AddRoundKey (all four operations)

Final Round N

SubBytes → ShiftRows → AddRoundKey (MixColumns omitted for structural symmetry with decryption)

Key Schedule: Expands original key into N+1 round keys. Each round consumes exactly one 128-bit round key.

AES Parameters: AES-128, AES-192, AES-256

All three variants use the same **128-bit block size** and the same round structure. They differ only in key size and round count — larger keys provide more security at the cost of additional rounds.

Parameter	AES-128	AES-192	AES-256
Key Size (bits)	128	192	256
Block Size (bits)	128	128	128
Number of Rounds	**10**	**12**	**14**
Round Keys	11	13	15
Key Schedule Words	44	52	60
Security Level	128-bit	192-bit	256-bit
Performance (SW)	Fastest	Moderate	~40% slower
Typical Use	Most applications	Intermediate security	Long-term classified data

CHAPTER 5

State Matrix Construction

How 16 plaintext bytes are loaded into the 4×4 working array that AES transforms through every round

Column-Major Order

Bytes are placed column by column, top to bottom, left to right — not row by row

4×4 Byte Matrix

State[row][col] — 4 rows × 4 columns = 16 bytes = 128 bits throughout all rounds

Working State

The matrix is modified in place by SubBytes, ShiftRows, MixColumns, AddRoundKey

Constructing the 4×4 State Matrix

Loading Plaintext into State

Plaintext: `00 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF` (16 bytes)

Arrangement: Bytes fill the state matrix **column by column** (top to bottom, left to right).

- **Column 0:** 00, 11, 22, 33
- **Column 1:** 44, 55, 66, 77
- **Column 2:** 88, 99, AA, BB
- **Column 3:** CC, DD, EE, FF

Index notation: State[0][0]=00, State[1][0]=11, State[2][0]=22, State[3][0]=33

Initial 4×4 State Matrix

Row\Col	Col 0	Col 1	Col 2	Col 3
Row 0	00	44	88	CC
Row 1	11	55	99	DD
Row 2	22	66	AA	EE
Row 3	33	77	BB	FF

This column-major ordering is critical — confusing it with row-major ordering is a common implementation error.

State Matrix: Byte-by-Byte Construction (Animated)

Each byte of the 16-byte plaintext is placed into the state matrix one at a time, filling column 0 first (bytes 0–3), then column 1 (bytes 4–7), and so on.

01

Byte 0 (0x00) → State[0][0]

First byte → Row 0, Column 0 (top-left)

02

Byte 1 (0x11) → State[1][0]

Second byte → Row 1, Column 0 (column continues downward)

03

Byte 2 (0x22) → State[2][0]

Third byte → Row 2, Column 0

04

Byte 3 (0x33) → State[3][0]

Column 0 complete; move to column 1

05

Bytes 4–7 (0x44–0x77) → Column 1

Byte 4 → State[0][1], Byte 5 → State[1][1], and so on

06

Bytes 8–15 → Columns 2 and 3

Same pattern fills the final two columns; state matrix is complete

The resulting 4×4 matrix of bytes is the **working state** throughout all AES rounds. Every operation modifies this matrix in place until the final ciphertext is read out.

CHAPTER 6

Key Expansion (Key Schedule)

From a single 128-bit key to 11 distinct 128-bit round keys — using RotWord, SubWord, and Rcon

1

Original Key

128 bits split into W[0]–W[3] (4 words)

2

RotWord + SubWord + Rcon

Applied at every 4th word boundary (i mod 4 = 0)

3

Simple XOR

W[i] = W[i-1] ⊕ W[i-4] for non-boundary words

4

44 Words Total

W[0]–W[43] → 11 round keys × 128 bits each

Key Schedule Overview: Generating 11 Round Keys

Algorithm Summary

Input: 128-bit key → split into 4 words W[0]–W[3]

Output: 44 words W[0]–W[43] → 11 round keys of 128 bits each

For i = 4 to 43:

- If i mod 4 ≠ 0: `W[i] = W[i−1] ⊕ W[i−4]`
- If i mod 4 = 0: `W[i] = W[i−4] ⊕ SubWord(RotWord(W[i−1])) ⊕ Rcon[i/4]`

The Three Sub-Operations

RotWord

Cyclic left rotation:
$[a_0,a_1,a_2,a_3] \rightarrow [a_1,a_2,a_3,a_0]$
Example: [09,CF,4F,3C] → [CF,4F,3C,09]

SubWord

Apply AES S-Box substitution to each of the 4 bytes independently

Rcon[i]

Round constant = $[x^{i-1}$ in $GF(2^8)$, 0x00, 0x00, 0x00] — breaks round symmetry

Rcon Table: Round Constants

Rcon[i] = [x^{i-1} mod m(x), 0x00, 0x00, 0x00] — Powers of x computed in GF(2^8). Purpose: break round symmetry to prevent related-key attacks.

i	x^{i-1} in GF(2^8)	Rcon[i][0]	Rcon[i] (full word)
1	$x^0 = 1$	0x01	[01, 00, 00, 00]
2	$x^1 = x$	0x02	[02, 00, 00, 00]
3	x^2	0x04	[04, 00, 00, 00]
4	x^3	0x08	[08, 00, 00, 00]
5	x^4	0x10	[10, 00, 00, 00]
6	x^5	0x20	[20, 00, 00, 00]
7	x^6	0x40	[40, 00, 00, 00]
8	x^7	0x80	[80, 00, 00, 00]
9	x^8 mod m(x)	0x1B	[1B, 00, 00, 00]
10	x^9 mod m(x)	0x36	[36, 00, 00, 00]

Key Expansion: Round Key 1 — Step-by-Step

Original Key (NIST FIPS 197): 2B 7E 15 16 28 AE D2 A6 AB F7 15 88 09 CF 4F 3C

Initial words: W[0]=2B7E1516, W[1]=28AED2A6, W[2]=ABF71588, W[3]=09CF4F3C

Computing W[4] (i=4, i mod 4 = 0)

01

RotWord(W[3])

[09,CF,4F,3C] → [CF,4F,3C,09]

02

SubWord

S(CF)=8A, S(4F)=84, S(3C)=EB, S(09)=01
→ [8A,84,EB,01]

03

XOR Rcon[1]

[8A,84,EB,01] ⊕ [01,00,00,00] = [8B,84,EB,01]

04

XOR W[0]

[8B,84,EB,01] ⊕ [2B,7E,15,16] = [A0,FA,FE,17]
W[4] = A0FAFE17 ✔️

Completing Round Key 1

W[5] = W[4] ⊕ W[1]
A0FAFE17 ⊕ 28AED2A6 = 88542CB1

W[6] = W[5] ⊕ W[2]
88542CB1 ⊕ ABF71588 = 23A33939

W[7] = W[6] ⊕ W[3]
23A33939 ⊕ 09CF4F3C = 2A6C7605

✓ **Round Key 1 = A0FAFE17 88542CB1 23A33939 2A6C7605**

CHAPTER 7

SubBytes

The only nonlinear operation in AES — every byte independently replaced via the 256-entry S-Box

Nonlinearity

Resists linear and differential cryptanalysis — critical for Shannon's confusion principle

Step 1: GF Inverse

Find a^{-1} in GF(2^8); 0x00 maps to 0x00 by convention

Step 2: Affine Transform

8×8 binary matrix multiplication + XOR with constant 0x63

Result: S-Box

256-entry lookup table; each input byte has a unique, fixed output

SubBytes: S-Box Construction

Two-Step Construction

01

Multiplicative Inverse in GF(2^8)

Find a^{-1} such that $a \cdot a^{-1} \equiv 1 \pmod{m(x)}$. Special case: 0x00 → 0x00.

02

Affine Transformation

Apply 8×8 circulant binary matrix multiplication plus XOR with constant c = 0x63 = 01100011_2.

Affine Transform Formula

$b'_i = b_i \oplus b_{(i+4)\%8} \oplus b_{(i+5)\%8} \oplus b_{(i+6)\%8} \oplus b_{(i+7)\%8} \oplus c_i$

Security Properties

Algebraic Complexity

The multiplicative inverse + affine transform creates a highly complex algebraic expression resistant to algebraic attacks

Optimal Nonlinearity

The S-Box achieves near-optimal nonlinearity metrics — differential uniformity = 4, nonlinearity = 112

Invertibility

Every S-Box output maps to exactly one input → the Inverse S-Box (IS-Box) is well-defined for decryption

SubBytes Example: 0x53 → 0xED

Step 1: Multiplicative Inverse

Input byte: $0x53 = 01010011_2$

Find $0x53^{-1}$ such that $0x53 \times ? \equiv 0x01 \pmod{m(x)}$

Using extended Euclidean algorithm applied to polynomials over GF(2):

$0x53^{-1} = 0xCA = 11001010_2$

Step 2: Affine Transformation

Apply 8×8 circulant matrix to $0xCA = [1,1,0,0,1,0,1,0]$

XOR each bit with its rotations and constant $0x63$

Result after affine transform: $0xED = 11101101_2$

Verification and Inverse

NIST Verification

S-Box[0x53] = 0xED ✔️
Confirmed in FIPS 197 Appendix A

Inverse S-Box

IS-Box[0xED] = 0x53
Used in AES decryption (InvSubBytes)

Special Case: 0x00

S-Box[0x00] = 0x63
Inverse of 0 defined as 0 by convention; affine transform gives 0x63

AES S-Box (Complete 16×16 Lookup Table)

Usage: S-Box[row][col] where row = upper nibble, col = lower nibble of input byte. **Examples:** S-Box[0x53] → row=5, col=3 → `0xED` | S-Box[0x00] = `0x63` | S-Box[0xFF] = `0x16`

Row	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF
0x	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1x	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2x	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3x	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75

Rows 0–3 shown (full table = 256 entries). The S-Box is fully specified in NIST FIPS 197 Appendix A.

CHAPTER 8

ShiftRows

Diffusion across rows — cyclic shifts that ensure bytes from every column spread to every other column

Row 0: Shift 0 No change — anchor row	**Row 1: Shift ← 1** One byte cyclic left
Row 2: Shift ← 2 Two bytes cyclic left	**Row 3: Shift ← 3** Three bytes cyclic left

ShiftRows: Cyclic Row Shifts

Purpose and Effect

ShiftRows ensures that bytes from each column spread to **different columns**, providing inter-column diffusion. Without ShiftRows, MixColumns would operate on each column independently, allowing parallel attacks.

Combined with MixColumns: After just 2 rounds, every output byte depends on every input byte — achieving **full diffusion**.

Inverse ShiftRows (decryption): Cyclic right shifts by 0, 1, 2, 3 bytes respectively — trivially reversed.

ShiftRows Before and After

Row	Before	Shift	After
Row 0	`S00 S01 S02 S03`	← 0	`S00 S01 S02 S03`
Row 1	`S10 S11 S12 S13`	← 1	`S11 S12 S13 S10`
Row 2	`S20 S21 S22 S23`	← 2	`S22 S23 S20 S21`
Row 3	`S30 S31 S32 S33`	← 3	`S33 S30 S31 S32`

After ShiftRows, **each column** of the new state contains bytes from **all 4 original columns** — the key to inter-column mixing.

CHAPTER 9

MixColumns

Maximum diffusion within each column — GF(2^8) matrix multiplication with the MDS property

MDS Matrix

Maximum Distance Separable — branch number 5 guarantees any 1 byte change affects ≥ 4 output bytes

GF(2^8) Arithmetic

All multiplications use xtime; ×03 = xtime(a) ⊕ a; ×01 = identity

Column-wise

Each of the 4 columns processed independently; outputs replace inputs in the state

MixColumns: Matrix Multiplication in GF(2^8)

The MDS Matrix

Each column treated as a 4-element vector, multiplied by the fixed AES MDS matrix over GF(2^8):

Row\Col	C0	C1	C2	C3
R0	02	03	01	01
R1	01	02	03	01
R2	01	01	02	03
R3	03	01	01	02

MDS Property and GF Multiplication

Branch Number = 5

Any single byte change in input results in at least 4 bytes changed in output — maximum possible diffusion for a 4×4 matrix

×02 Multiplication

$02 \cdot a = \text{xtime}(a)$ — single left-shift with conditional 0x1B reduction

×03 Multiplication

$03 \cdot a = \text{xtime}(a) \oplus a$ — shift then add original (distributive property)

MixColumns Example: Column [d4, bf, 5d, 30]

Input column from NIST FIPS 197: [d4, bf, 5d, 30]

Computing Output Byte 0

$b'_0 = 02 \cdot d4 \oplus 03 \cdot bf \oplus 01 \cdot 5d \oplus 01 \cdot 30$

02·d4: d4=11010100, bit7=1 → shift=10101000, XOR 1B → 10110011 = b3

03·bf: xtime(bf): bf=10111111, bit7=1 → shift=01111110, XOR 1B → 65; then 65⊕bf=da

01·5d = 5d (identity); **01·30 = 30**

Result: b3 ⊕ da ⊕ 5d ⊕ 30 = 04 ✔️

Full Column Output

Output	Calculation	Result
b'_0	02·d4⊕03·bf⊕5d⊕30	

MixColumns: Complete Column Calculation Table

Input: [d4, bf, 5d, 30] | All GF(2^8) multiplications performed using xtime

xtime values: xtime(d4)=b3 | xtime(bf)=65 | xtime(5d)=ba | xtime(30)=60

×03 values: 03·d4=b3⊕d4=67 | 03·bf=65⊕bf=da | 03·5d=ba⊕5d=e7 | 03·30=60⊕30=50

Output Byte	02×	03×	01× (3rd)	01× (4th)	XOR Result
b'_0	02·d4=b3	03·bf=da	01·5d=5d	01·30=30	

CHAPTER 10

AddRoundKey

The key-mixing step — XOR every state byte with the corresponding round key byte

Simplest Operation

Bitwise XOR — but the only step that introduces the secret key into the state

Self-Inverse

XOR with the same key twice returns the original: (a ⊕ k) ⊕ k = a

Applied 11 Times

Once at Round 0 (with original key) and once after each of the 10 rounds

AddRoundKey: Bit-by-Bit XOR

Operation Detail

XOR each byte of the state matrix with the corresponding byte of the current round key. All 16 bytes are processed simultaneously.

Single Byte Example: 0x32 ⊕ 0x2B

$0x32 = 00110010_2$
$0x2B = 00101011_2$
$XOR = 00011001_2 = 0x19$

Security

Without the key, the XOR output is **uniformly random** — provides key-dependent transformation. Each of the 11 round keys is unique, so the cipher state is re-randomized at every round boundary.

AddRoundKey XOR Example (Round 1)

Position	State Byte	Round Key	XOR Result
[0][0]	04	A0	

AddRoundKey: Full State Matrix Example

Round Key 1: `A0 FA FE 17 | 88 54 2C B1 | 23 A3 39 39 | 2A 6C 76 05`

XOR is applied element-wise to every position in the 4×4 state. The round key is arranged in the same 4×4 column-major layout as the state.

State Before AddRoundKey

R\C	C0	C1	C2	C3
R0	04	E0	48	28
R1	66	CB	F8	06
R2	81	19	D3	26
R3	E5	9A	7A	4C

State After AddRoundKey (Round 1 Output)

R\C	C0	C1	C2	C3
R0	A4	68	6B	02
R1	9C	9F	D4	B7
R2	7F	BA	EA	1F
R3	F2	F6	0D	49

This state is the input to Round 2 SubBytes.

CHAPTER 11

Complete AES-128 Encryption

Round 0 through Round 10 — NIST FIPS 197 official test vectors traced step by step

Plaintext

`32 43 F6 A8 88 5A 30 8D 31 31 98 A2 E0 37 07 34`

Key

`2B 7E 15 16 28 AE D2 A6 AB F7 15 88 09 CF 4F 3C`

Ciphertext

`39 25 84 1D 02 DC 09 FB DC 11 85 97 19 6A 0B 32`

Complete Encryption: NIST Test Vector Setup

NIST FIPS 197 Test Vectors

Plaintext:
32 43 F6 A8 88 5A 30 8D
31 31 98 A2 E0 37 07 34

Key:
2B 7E 15 16 28 AE D2 A6
AB F7 15 88 09 CF 4F 3C

Expected Ciphertext:
39 25 84 1D 02 DC 09 FB
DC 11 85 97 19 6A 0B 32

10-Round Encryption Structure

01

Round 0 (Initial)

AddRoundKey: State = Plaintext ⊕ Original Key

02

Rounds 1–9

SubBytes → ShiftRows → MixColumns → AddRoundKey (with round key r)

03

Round 10 (Final)

SubBytes → ShiftRows → AddRoundKey(RK10) — no MixColumns

State matrix tracked after every individual operation — 40+ intermediate states verifiable against FIPS 197 Appendix B.

Round 0: Initial AddRoundKey

Operation: State = Plaintext state matrix ⊕ Original Key (W[0]–W[3])

Byte-by-Byte XOR (Column 0 example)

32 ⊕ 2B = 19
88 ⊕ 28 = A0
31 ⊕ AB = 9A
E0 ⊕ 09 = E9

43 ⊕ 7E = 3D
5A ⊕ AE = F4
31 ⊕ F7 = C6
37 ⊕ CF = F8

F6 ⊕ 15 = E3
30 ⊕ D2 = E2
98 ⊕ 15 = 8D
07 ⊕ 4F = 48

A8 ⊕ 16 = BE
8D ⊕ A6 = 2B
A2 ⊕ 88 = 2A
34 ⊕ 3C = 08

State After Round 0 (Initial AddRoundKey)

R\C	Col 0	Col 1	Col 2	Col 3
Row 0	19	A0	9A	E9
Row 1	3D	F4	C6	F8
Row 2	E3	E2	8D	48
Row 3	BE	2B	2A	08

This state is the input to Round 1 SubBytes.

Rounds 1–9: Iterative Round Operations

Round 1 Walkthrough

After SubBytes: Each byte replaced via S-Box lookup
Example: S-Box[0x19] = 0xD4, S-Box[0x3D] = 0x27, ...

After ShiftRows: Rows cyclically shifted
Row 0 unchanged; Row 1 left by 1; Row 2 left by 2; Row 3 left by 3

After MixColumns: Columns mixed via GF(2^8) matrix multiply — column [d4,bf,5d,30] → [04,66,81,e5]

After AddRoundKey (RK1): XOR with `A0FAFE17 88542CB1 23A33939 2A6C7605`

Round 1 Final State

R\C	C0	C1	C2	C3
R0	A4	9C	7F	F2
R1	3D	4D	C0	2F
R2	58	11	31	99
R3	AD	0C	47	04

Avalanche effect: By Round 2, a single bit change in plaintext affects ~50% of all state bits — demonstrating AES's exceptional diffusion.

Round 10: Final Round (No MixColumns)

Final Round Structure

01

SubBytes

Apply S-Box to all 16 bytes of the Round 9 output state

02

ShiftRows

Cyclic row shifts identical to all previous rounds (0, 1, 2, 3)

03

AddRoundKey

XOR with Round Key 10 — no MixColumns in the final round

Why No MixColumns?

MixColumns would need to be inverted at the start of decryption. Omitting it from Round 10 makes the final round structurally symmetric with the initial round, simplifying both implementation and proof of correctness.

Final Ciphertext

R\C	C0	C1	C2	C3
R0	39	02	DC	19
R1	25	DC	11	6A
R2	84	09	85	0B
R3	1D	FB	97	32

Ciphertext: 39 25 84 1D 02 DC 09 FB DC 11 85 97 19 6A 0B 32 ✔️
Matches NIST FIPS 197 Appendix B exactly.

CHAPTER 12

AES Decryption

Inverse operations in reverse round order — every encryption step has an exact algebraic inverse

1

Ciphertext

128-bit input; same key used as encryption

2

Reverse Rounds 10→1

InvSubBytes, InvShiftRows, InvMixColumns, AddRoundKey

3

AddRoundKey 0

Final XOR with original key

4

Plaintext

Exact original 128-bit plaintext recovered

Decryption Overview: Reverse Order, Inverse Operations

Decryption Round Order

01

Round 10 (first in decryption)

AddRoundKey(RK10) → Inv ShiftRows → Inv SubBytes

02

Rounds 9–1

AddRoundKey(RKr) → Inv MixColumns → Inv ShiftRows → Inv SubBytes

03

Round 0 (final)

AddRoundKey(RK0) — with original key → Plaintext

Inverse Operation Properties

InvSubBytes

Apply Inverse S-Box (IS-Box) — reverses affine transform then takes $GF(2^8)$ inverse

InvShiftRows

Cyclic **right** shifts by 0, 1, 2, 3 bytes — trivially reverses left shifts

InvMixColumns

Multiply by the inverse MDS matrix over $GF(2^8)$ — requires ×09, ×0B, ×0D, ×0E

AddRoundKey (self-inverse)

XOR with same round key reverses the operation: $(a \oplus k) \oplus k = a$

Inverse MixColumns Matrix

Inverse MDS Matrix

R\C	C0	C1	C2	C3
R0	0E	0B	0D	09
R1	09	0E	0B	0D
R2	0D	09	0E	0B
R3	0B	0D	09	0E

Verification: Forward × Inverse = Identity matrix over GF(2^8) ✔️

Computing Inverse Multiplications via xtime

0E · a

xtime(xtime(xtime(a))) ⊕ xtime(xtime(a)) ⊕ xtime(a)

0B · a

xtime(xtime(xtime(a))) ⊕ xtime(a) ⊕ a

0D · a

xtime(xtime(xtime(a))) ⊕ xtime(xtime(a)) ⊕ a

09 · a

xtime(xtime(xtime(a))) ⊕ a

Inverse SubBytes: IS-Box Lookup

IS-Box reverses the AES S-Box: applies the inverse affine transform first, then finds the multiplicative inverse in GF(2^8). **IS-Box[0xED] = 0x53** (reverses S-Box[0x53] = 0xED).

Row	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF
0x	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
1x	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
2x	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
3x	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25

IS-Box rows 0–3 shown. Inverse affine transform constant: `0x05` (vs. `0x63` in forward direction). Full IS-Box specified in NIST FIPS 197.

CHAPTER 13

AES Implementation in Python

Translating the mathematics directly into clean, readable code — every function maps to an AES operation

Python AES: Core Functions Pseudocode

GF(2^8) Primitives

```
def xtime(a):
    # Multiply by x (0x02) in GF(2^8)
    if a & 0x80:
        return ((a << 1) ^ 0x1B) & 0xFF
    else:
        return (a << 1) & 0xFF

def gmul(a, b):
    # GF(2^8) multiplication
    # Russian peasant algorithm
    p = 0
    while b:
        if b & 1: p ^= a
        a = xtime(a)
        b >>= 1
    return p
```

Round Operations

```
def sub_bytes(state):
 for i in range(4):
 for j in range(4):
 state[i][j] = S_BOX[state[i][j]]

def shift_rows(state):
 for r in range(1, 4):
 state[r] = state[r][r:]+state[r][:r]

def add_round_key(state, rk):
 for i in range(4):
 for j in range(4):
```